

AI-Powered Linux Hardening Assistant - Master Prompt

You are a Senior Full Stack Engineer specializing in Cybersecurity, FastAPI, React, PostgreSQL, Docker, Paramiko, and Google Gemini.

Objective

Create a system that: 1. Accepts Linux server credentials. 2. Connects via SSH. 3. Executes predefined Linux security audit commands. 4. Aggregates outputs into a single report. 5. Sends the report to Google Gemini for analysis. 6. Receives structured security findings. 7. Stores audit results in PostgreSQL. 8. Displays findings in a React dashboard.

Tech Stack

Backend: FastAPI, PostgreSQL, SQLAlchemy, Paramiko

Frontend: React, Vite, Tailwind CSS, Axios

AI: Google Gemini API

Infrastructure: Docker, Docker Compose

Audit Commands

```
grep PermitRootLogin /etc/ssh/sshd_config
grep PasswordAuthentication /etc/ssh/sshd_config
grep MaxAuthTries /etc/ssh/sshd_config
grep PASS_MAX_DAYS /etc/login.defs
ufw status
whoami
hostname
uname -a
systemctl is-active ssh
cat /etc/os-release
```

Aggregate all outputs into a single audit report.

Gemini Instructions

Analyze the report and identify: - Security misconfigurations - Weak settings - Hardening opportunities Classify findings as: - Critical - High - Medium - Low Rule: PermitRootLogin enabled = Critical

Return JSON: { "findings": [{ "severity": "", "issue": "", "explanation": "", "impact": "", "fix_command": "" }] }

Database Schema

audits: - id - server_ip - audit_date - raw_report findings: - id - audit_id - severity - issue - explanation - impact - fix_command

Backend Services

- ssh_service.py - audit_service.py - gemini_service.py - database_service.py

API Endpoints

POST /audit
GET /audits
GET /audit/{id}

Frontend Pages

- Login - New Audit - Audit Results - Audit History

Dashboard Features

- Critical Count - High Count - Medium Count - Low Count Findings Table: - Issue - Severity - Explanation - Impact - Fix Command

Scheduling

Support scheduled audits using APScheduler. Queue audit jobs. Prevent concurrent SSH execution. Support configurable schedules.

Code Requirements

- Clean Architecture - SOLID Principles - Repository Pattern - Service Layer Pattern - Dependency Injection - Type Hints - Structured Logging - Robust Error Handling - Production-Ready Code

Deliverables

For every phase provide: 1. Updated File Tree 2. Complete Source Code 3. Configuration Files 4. Documentation Updates 5. Testing Instructions Generate only the requested phase. Do not generate future phases unless explicitly requested.